PAC-MAN REACTIVE APPROACH

Intelligence Autonomous Agent course

APRIL 6, 2026

AUTHORS: MICHAEL O'SHEA (2025244317) MOHAMMED ABDELQADER (2025179643)

1. Introduction

This project focuses on developing a reactive agent to control Pac-Man based on the requirements of the assignment. The goal is to design a production system where actions are selected using current perceptions such as ghosts, food, and walls. Three reactive agents were implemented in agents.py: a random baseline agent, a rule-based agent using legal movements, and an improved version that also chases frightened ghosts. These agents represent different strategies, progressing from simple behavior to more effective decision-making.

2. Experimental Setup

The experimental setup followed the guidelines provided in the assignment. Each agent configuration was executed 100 times using different random seeds to assess robustness and consistency. The results were recorded and stored for subsequent analysis, as discussed later in this report. To improve computational efficiency, multiprocessing was employed to run multiple agent instances in parallel.

3. The actual Agents

3.1 pacman_reactive_agent_no_random_legal

The agent uses local perception to detect ghosts, food (dots), and walls in all four directions within a short range. It relies on wall detection to determine which movements are legal, ensuring that Pacman does not attempt to move into blocked paths. Based on these perceptions, the agent prioritizes avoiding nearby ghosts, then moves toward detected food if it is safe, and finally selects a valid direction that does not reverse its movement when no other information is available. The agent uses a small amount of memory to determine the opposite direction and will use that to avoid oscillations.

Perceptions	Actions
Ghost up	Move down
Ghost down	Move up
Ghost left	Move right
Ghost right	Move left
Wall up	
Wall down	
Wall left	
Wall right	
Dot up	
Dot down	
Dot left	
Dot right	
Memory(Opposite Direction)	

Production rules(table):		
ID	**Perception**	**Actions**
R1	If Ghost up and not wall down	Move Down
R2	If Ghost down and not wall up	Move Up
R3	If Ghost left and not wall right	Move Right
R4	If Ghost right and not wall left	Move Left
R5	If Dot up and not wall up	Move Up
R6	If Dot down and not wall down	Move Down
R7	If Dot left and not wall left	Move Left
R8	If Dot right and not wall right	Move Right
R9	If wall up and not M(opposite direction) up	Move Up
R10	If wall down and not M(opposite direction) down	Move Down
R11	If wall left and not M(opposite direction) left	Move Left
R12	If wall right and not M(opposite direction) right	Move Right

Figure 1

3.2 pacman_reactive_agent_no_random_legal_chaseghosts

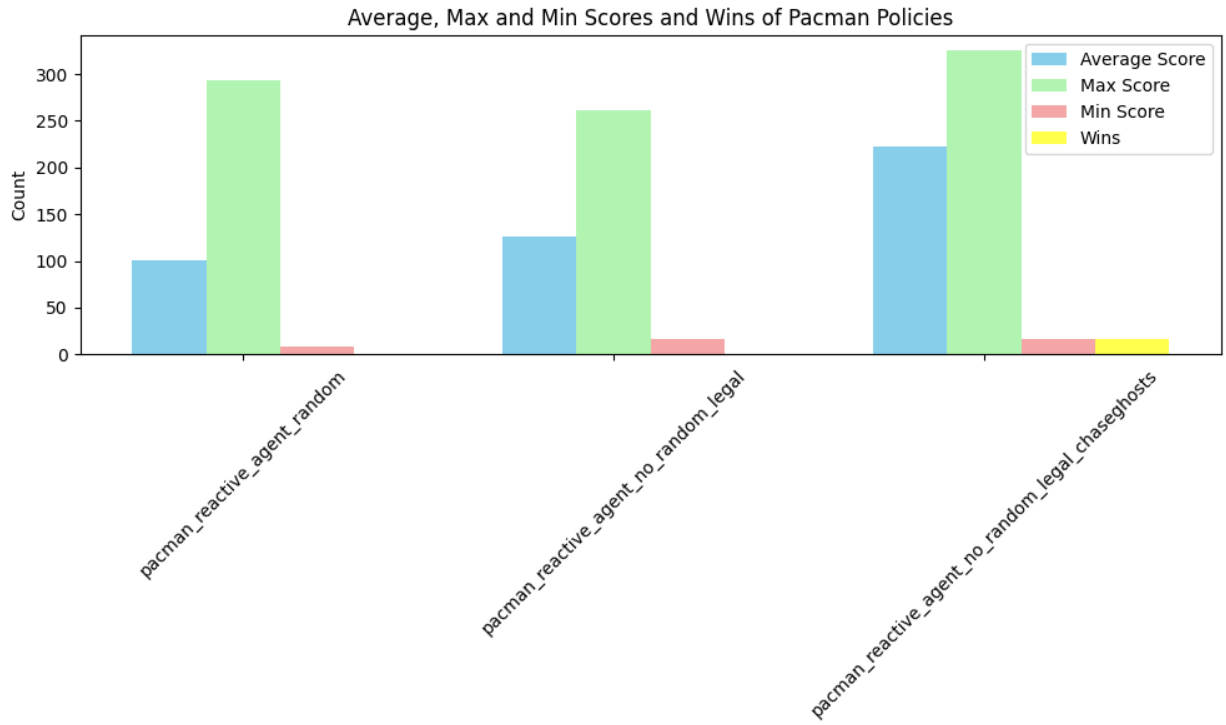
The agent operates using local perception to detect active ghosts, frightened ghosts, food, and walls at different ranges. It first prioritizes safety by avoiding active ghosts using valid movements determined by wall detection. If no immediate danger is present, the agent switches to an aggressive behavior by chasing nearby frightened ghosts to gain points. If neither threat nor opportunity is detected, it moves toward food within a larger range. In cases where no specific target is perceived, the agent selects a valid direction while avoiding reversing its movement unless necessary. This results in a reactive agent that dynamically balances survival, reward maximization, and efficient navigation based on its current perceptions. The agent uses a small amount of memory to determine the opposite direction and will use that to avoid oscillations and also uses a small amount of memory to store the detection range of active ghosts, frightened ghosts, and Food.

Perceptions	Actions
Ghost up	Move down
Ghost down	Move up
Ghost left	Move right
Ghost right	Move left
Wall up	
Wall down	
Wall left	
Wall right	
Dot up	
Dot down	
Dot left	
Dot right	
Memory(Opposite Direction)	
Memory(Active Ghost Detection Range)	
Memory(Frightened Ghost Detection Range)	
Memory(Food Detection Range)	

Production rules(table):		
ID	**Perception**	**Actions**
R1	If Ghost up within range M(Active) and not Frightened ghost up within range M(Active) and not wall down	Move Down
R2	If Ghost down within range M(Active) and not Frightened ghost down within range M(Active) and not wall up	Move Up
R3	If Ghost left within range M(Active) and not Frightened ghost left within range M(Active) and not wall right	Move Right
R4	If Ghost right within range M(Active) and not Frightened ghost right within range M(Active) and not wall left	Move Left
R5	If Ghost Frightened up within range M(Frightened) and not wall up	Move Up
R6	If Ghost Frightened down within range M(Frightened) and not wall down	Move Down
R7	If Ghost Frightened left within range M(Frightened) and not wall left	Move Left
R8	If Ghost Frightened right within range M(Frightened) and not wall right	Move Right
R9	If Food up within range M(Food) and not wall up	Move Up
R10	If Food down within range M(Food) and not wall down	Move Down
R11	If Food left within range M(Food) and not wall left	Move Left
R12	If Food right within range M(Food) and not wall right	Move Right
R13	If wall up and not M(opposite direction) up	Move Up
R14	If wall down and not M(opposite direction) down	Move Down
R15	If wall left and not M(opposite direction) left	Move Left
R16	If wall right and not M(opposite direction) right	Move Right

Figure 2

4. Results



Policy Name	Max Score	Min Score	Average Score	Standard Dev	Wins
pacman_reactive_agent_random	293.0	8.0	101.25	88.71	0
pacman_reactive_agent_no_random_legal	267.0	16.0	126.62	44.95	0
pacman_reactive_agent_no_random_legal_chaseghosts	325.0	16.0	222.83	64.99	17

Figure 3

The results in the table (figure 3) indicate that all agents outperform the random baseline in terms of average score, maximum score, and variability. In particular, tuning the distance hyperparameters in the chase-ghosts model led to an increase in maximum score while also enabling occasional successful game completions.

5. Additional Work

We also implemented additional agents to explore the potential performance improvements achievable through more complex strategies, including pathfinding, advanced ghost prediction, and dead-end detection. These agents, however, utilize information that is not accessible through standard perceptions, violating the project constraints. As such, they are presented only to illustrate the upper bounds of performance and are excluded from the main benchmark due to lack of comparability with perception-limited agents.

Policy Name	Max Score	Min Score	Average Score	Standard Dev	Wins
pacman_reactive_agent_no_random_mark1	259.0	22.0	217.34	76.58	85
pacman_reactive_agent_no_random_mark2	282.0	22.0	133.59	48.46	0
pacman_reactive_agent_no_random_mark3	279.0	22.0	227.81	82.65	82

Figure 4

Further details can be found in agents.py. Briefly, Mark1 prioritizes food collection, Mark2 introduces ghost-chasing behavior to increase score but performs sub-optimally, and Mark3 refines this approach by implementing a more intelligent chasing strategy, resulting in improved win rates while maintaining strong scoring performance.

5.1 The WIN vs Max Score dilemma

Depending on the chosen strategy, you may either achieve a higher win rate or a higher score, but these are usually inversely related. A more conservative, risk-averse strategy focused on collecting food tends to increase the chances of winning, but results in lower scores. In contrast, a more aggressive strategy that involves chasing ghosts can lead to higher scores, but often reduces the overall win rate. This trade-off is reflected in the results shown above in Figure 4.

5.2 Manhattan vs Maze distance discussion

The provided game engine computes distances using Manhattan distance, which ignores obstacles such as walls. This limitation led to issues in the Mark1 agent, where Pacman could become trapped in local loops while navigating around obstacles without making real progress toward food. Two potential solutions were considered: incorporating memory to prevent revisiting recent positions, or replacing Manhattan distance with maze distance. The latter is more effective, as maze distance applies a breadth-first search algorithm that accounts for walls, producing a more accurate representation of actual path distance.

6. Conclusions

In this project, a reactive Pac-Man agent was successfully designed and implemented using a production system based on local perceptions such as ghosts, food, and walls. Multiple agent strategies were developed, progressing from a random baseline to more structured and intelligent behaviors. The results demonstrate that incorporating rule-based decision-making significantly improves performance, while more advanced strategies, such as chasing frightened ghosts, can further increase scores but introduce trade-offs in win rate. Overall, the project highlights the effectiveness of reactive agents in dynamic environments and shows how small improvements in perception and decision rules can lead to substantial performance gains.

Remark: ChatGPT assisted only with report wording and played no role in the core analysis or results.